

University Management System Architecture

This repository contains the arc42 architecture documentation for the University Management System (UMS).

Chapters

- | | |
|-----------------------------|-------------------------------|
| 1. Introduction and Goals | 7. Deployment View |
| 2. Architecture Constraints | 8. Cross-cutting Concepts |
| 3. Context and Scope | 9. Architecture Decisions |
| 4. Solution Strategy | 10. Quality Requirements |
| 5. Building Block View | 11. Risks and Technical Debts |
| 6. Runtime View | 12. Glossary |

Table of Contents

1 Introduction and Goals

- 1.1 Requirements Overview
- 1.2 Quality Goals
- 1.3 Stakeholders

2 Architecture Constraints

- 2.1 Technical Constraints
- 2.2 Organizational Constraints
- 2.3 Development Constraints

3 Context and Scope

- 3.1 Business Context
- 3.2 Technical Context
 - 3.2.1 Mapping of Inputs/Outputs to Channels

4 Solution Strategy

- 4.1 Usability
- 4.2 Security
- 4.3 Reliability
- 4.4 Maintainability and Extensibility

5 Building Block View

- 5.1 Level 1 — Overall System
 - 5.1.1 White-box: University Management System
- 5.2 Level 2 — Student Management
 - 5.2.1 White-box: Student Management module
- 5.3 Level 3 — StudentController, StudentService, StudentRepository

6 Runtime View

- 6.1 Runtime Participants
- 6.2 Grade Notification Cannot Be Delivered
 - 6.2.1 Initial Situation
 - 6.2.2 Runtime Steps
 - 6.2.3 Architectural Decision
 - 6.2.4 Expected Result
- 6.3 Payment Result Is Unknown
 - 6.3.1 Initial Situation
 - 6.3.2 Runtime Steps
 - 6.3.3 Architectural Decision and CAP
 - 6.3.4 Expected Result
- 6.4 Summary

7 Deployment View

- 7.1 Infrastructure Level 1
 - 7.1.1 Overview Diagram
 - 7.1.2 Motivation
 - 7.1.3 Quality and/or Performance Features
 - 7.1.4 Mapping of Building Blocks to Infrastructure
- 7.2 Infrastructure Level 2
 - 7.2.1 Application Server
 - 7.2.2 Database Server

8 Cross-cutting Concepts

- 8.1 Authentication and Authorization
- 8.2 Input Validation and Error Handling
- 8.3 Logging and Monitoring
- 8.4 Data Persistence and Consistency
- 8.5 Scalability and Deployment

9 Architecture Decisions

10 Quality Requirements

10.1 Quality Requirements Overview

10.1.1 Quality Tree

10.2 Quality Scenarios

10.2.1 QS-1: Course Enrollment

10.2.2 QS-2: Unauthorized Access to Grades

10.2.3 QS-3: Failure During Grade Publication

10.2.4 QS-4: Lost FMS Response

11 Risks and Technical Debts

11.1 Risk Priority

12 Glossary

1 Introduction and Goals

1.1 Requirements Overview

The University Management System (UMS) is a web-based system that supports and simplifies important university processes.

At the moment, many tasks are still handled manually using spreadsheets and emails. This takes a lot of time, increases the chance of mistakes, and makes communication more difficult than necessary.

The goal of the system is to bring these processes together in one central platform for students, lecturers, and administrators.

The main features of the system are:

- course and curriculum management
- enrollment management
- attendance tracking
- grading and transcript management
- billing and payment management
- reporting and analytics
- user authentication and authorization
- access to course materials

The most important use cases are:

- students enroll in courses
- lecturers track attendance
- lecturers manage grades
- students view course material
- administrators generate reports

Overall, the system should reduce manual work, improve communication, and make university processes more efficient and reliable.

The requirements summarized in this section are based on the Business Requirements Document, the Use Case Document, and the stakeholder interview.

1.2 Quality Goals

Priority	Quality Goal	Motivation / Scenario
1	Usability	The system will be used by students, lecturers, and administrators. It should be easy to understand and simple to use for all user groups.

Priority	Quality Goal	Motivation / Scenario
2	Security	The system handles sensitive information such as student data, grades, and attendance records. This data must be protected and only be accessible to authorized users.
3	Reliability	The system should work correctly and be available during important processes such as course enrollment, attendance tracking, and grade submission.

These goals are made concrete through measurable scenarios in [Section 10: Quality Requirements](#).

1.3 Stakeholders

Role/Name	Contact	Expectations
John Smith, University Administrator	University Administration	Needs a reliable system that supports administrative work and reduces manual effort.
Michael Lee, Chief Information Officer	University IT Department	Expects a secure system that can integrate with existing university systems and support long-term operation.
Jane Doe, Senior Lecturer	Faculty	Needs support for grading, attendance tracking, easier communication with students, and less manual administrative work.
Emily Chen, Student Representative	Students	Expects a user-friendly system for enrollment, course materials, grades, and notifications.
Sarah Johnson, Financial Officer	Finance Department	Needs correct and reliable handling of billing and financial data.
Project Team	Development Team	Needs clear requirements, understandable documentation, and a realistic project scope.

2 Architecture Constraints

2.1 Technical Constraints

Constraint	Explanation
Frontend technology: React	The frontend must be implemented using React.
Backend technology: Node.js	Backend services must be implemented using Node.js.
Database: MongoDB	MongoDB must be used as the primary data store.
Cloud hosting	The application must be deployable on AWS.
Existing system integration	Integration with the Student Information System and Financial Management System is required.
Data protection compliance	The system must comply with applicable privacy and data protection regulations.

2.2 Organizational Constraints

Constraint	Explanation
Multiple user groups	The system must support students, lecturers, administrators, and financial staff.
Web-based solution	The application must be accessible through a standard web browser.
Centralized platform	Academic, administrative, and financial processes shall be managed in one system.

2.3 Development Constraints

Constraint	Explanation
Role-based access control	Authentication and authorization are mandatory.
Modular architecture	The system should be structured into independent functional modules.
Scalability	The architecture must support growing numbers of users and data.
Maintainability	Future extensions should be possible with minimal impact on existing functionality.

These constraints are reflected in the [Solution Strategy](#), the [Building Block View](#), and the [Cross-cutting Concepts](#).

3 Context and Scope

3.1 Business Context

The University Management System receives and sends data from and to the following communication partners:

University Management System

Communication Partner	Input (received by UMS)	Output (sent by UMS)
Students	Enrollment requests, attendance confirmations, payment submissions	Course schedules, grades, transcripts, course materials, notifications
Lecturers	Attendance records, grade submissions, uploaded course materials	Class lists, attendance summaries, grading confirmations
Administrators	Scheduling inputs, user account management actions, report requests	Generated reports, scheduling confirmations, system alerts
Finance Staff	Billing configurations, payment verifications	Invoices, payment confirmations, financial reports
Student Information System (SIS)	Student master data, registration records	Updated enrollment status, course assignments
Financial Management System (FMS)	Financial transaction records, payment status updates	Billing records, payment requests
Email Service	Delivery status or errors	Student notifications

All human actors interact with the system through a browser-based interface. No specialized client software is required. The SIS, FMS, and Email Service are external systems that the UMS integrates with through dedicated interfaces.

The internal modules responsible for these interactions are described in the [Building Block View](#). Relevant failure behavior for the FMS and Email Service is shown in the [Runtime View](#).

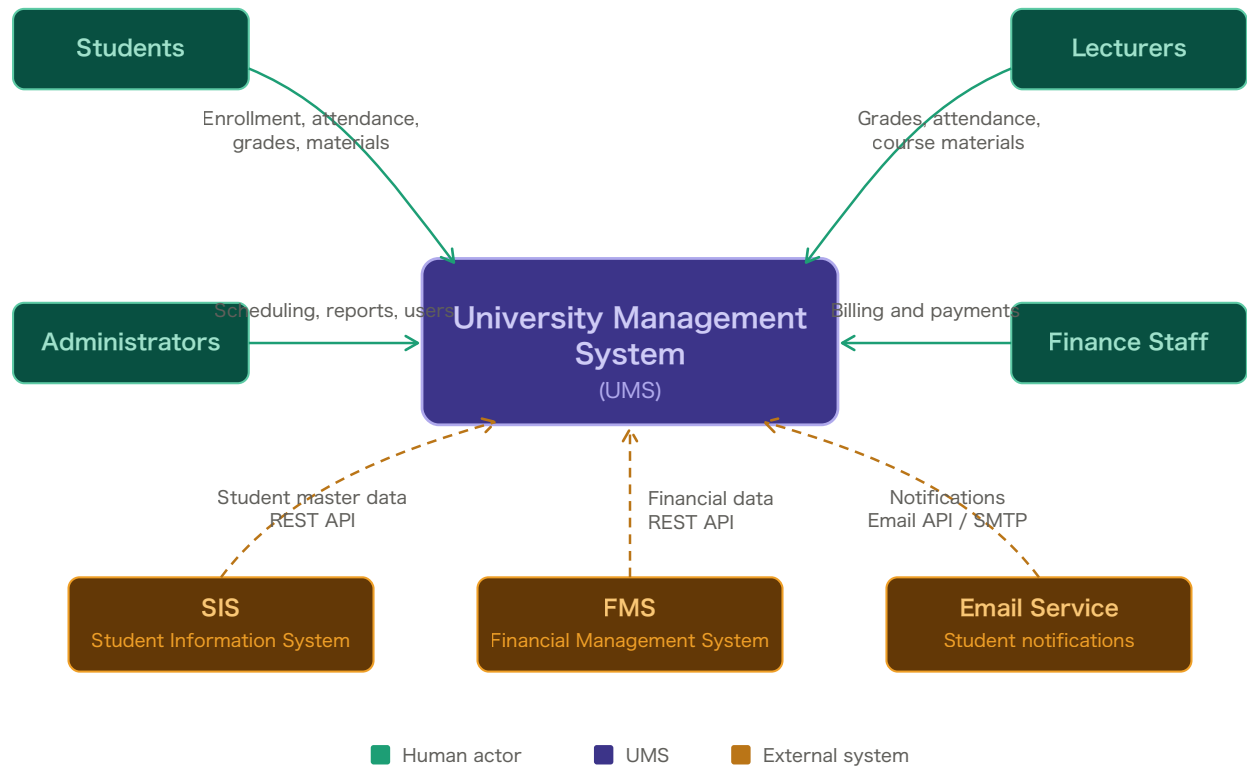


Figure 3.1: Business Context of the University Management System

3.2 Technical Context

The UMS communicates with its environment through the following technical channels:

Channel	Technology	Protocol	Description
User interface	React frontend, standard web browser	HTTPS	All human actors access the system through a browser. No client installation is required.
UMS ↔ SIS	REST API	HTTPS	The UMS exchanges student data with the existing Student Information System through a documented REST API.
UMS ↔ FMS	REST API	HTTPS	The UMS exchanges financial and billing data with the existing Financial Management System through a documented REST API.
UMS ↔ Email Service	Email API or SMTP	HTTPS / SMTP	The UMS sends notifications to students through an external email service.
UMS backend ↔ database	MongoDB driver	Internal TCP	The Node.js backend communicates with the MongoDB database within the cloud infrastructure. The database is not exposed externally.

Channel	Technology	Protocol	Description
Cloud infrastructure	AWS	—	The application is deployed and hosted on AWS. All communication between components is secured within the cloud environment.

3.2.1 Mapping of Inputs/Outputs to Channels

Input / Output	Channel	Protocol	Notes
All user interactions (students, lecturers, admins, finance staff)	Browser → React Frontend → Node.js Backend	HTTPS	Encrypted in transit; accessible from any standard browser
Grade submissions, attendance records, enrollment requests	User browser → Backend API	HTTPS / REST	Authenticated via role-based access control
Course materials, schedules, notifications	Backend → User browser	HTTPS / REST	Served through the React frontend
Email notifications	Backend → Email Service	HTTPS / SMTP	Sent asynchronously after the related operation succeeds
Student master data (read/write)	Backend ↔ Student Information System	HTTPS / REST API	Synchronized through a documented external API
Billing and payment data (read/write)	Backend ↔ Financial Management System	HTTPS / REST API	Synchronized through a documented external API
Persistent application data (all modules)	Backend ↔ MongoDB	Internal TCP	Database is not exposed outside the cloud environment

The physical placement of these elements is documented in the [Deployment View](#).

4 Solution Strategy

The University Management System (UMS) will be implemented as a centralized web-based platform that combines academic, administrative, and financial processes within a single application.

The architecture follows a modular monolith approach. This allows the system to remain easy to develop, deploy, and maintain while still separating the major business domains such as student management, course management, attendance tracking, grading, billing, and reporting. The resulting module structure is documented in the [Building Block View](#), and the decision is recorded in the [Architecture Decisions](#).

The following strategies are used to achieve the project's [quality goals](#):

4.1 Usability

- Provide a browser-based user interface that is accessible from different devices.
- Offer dashboards with role-specific information for students, lecturers, administrators, and finance staff.
- Design intuitive workflows for common tasks such as enrollment, attendance tracking, grade submission, and payment processing.

4.2 Security

- Implement role-based access control to ensure that users can only access information relevant to their responsibilities.
- Use secure authentication mechanisms for all users.
- Protect sensitive information such as grades, attendance records, and billing data.
- Comply with applicable data protection and privacy regulations.

4.3 Reliability

- Deploy the application on AWS as a scalable cloud platform.
- Use a centralized database to ensure data consistency and integrity.
- Provide controlled access to business functions through clearly defined application services.
- Integrate existing university systems through stable and documented APIs.

4.4 Maintainability and Extensibility

- Separate business domains into independent modules.
- Apply clear architectural boundaries between presentation, business logic, and data access layers.
- Design the system so that future modules and university processes can be added with minimal impact on existing functionality.

Overall, the chosen solution strategy supports the project goals of reducing manual work, minimizing human errors, improving communication between stakeholders, and providing a reliable platform for university administration.

5 Building Block View

5.1 Level 1 — Overall System

5.1.1 White-box: University Management System

The University Management System is decomposed into eight main functional building blocks. Each building block is responsible for a clearly defined area. All functional modules use the Authentication building block for access control. Technical and external supporting elements, such as the Message Queue and Email Service, are connected through the interfaces described below.

The interaction of these elements during grade publication and payment processing is documented in the [Runtime View](#).

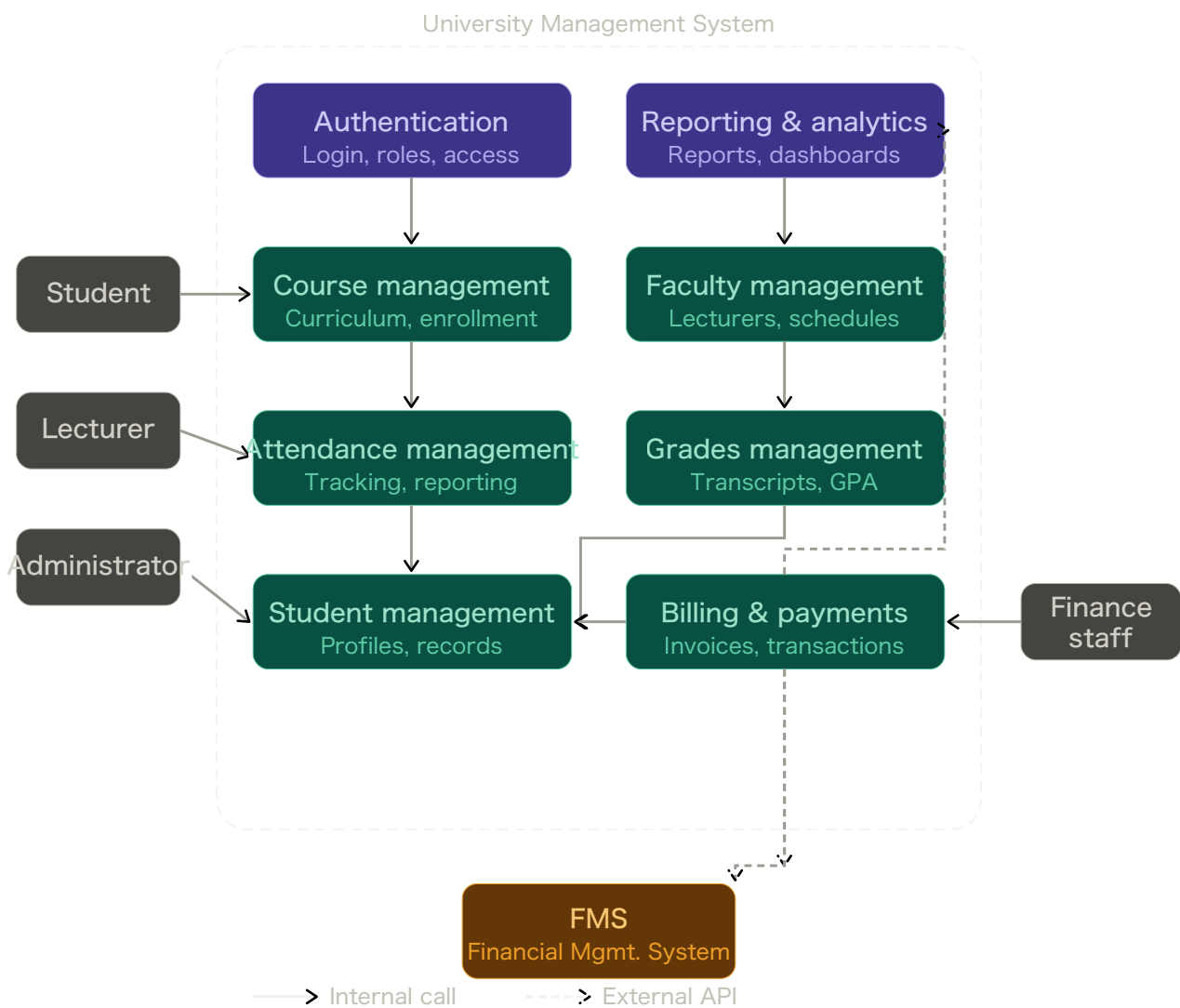


Figure 5.1: Level 1 Building Block View of the University Management System

5.1.1.1 Contained building blocks

Building Block	Responsibility
Authentication	Manages user login, session handling, and role-based access control for all user groups.
Course management	Manages the course catalog, curriculum, and student enrollment into courses.
Faculty management	Manages lecturer profiles, teaching assignments, and department scheduling.
Attendance management	Tracks and reports student attendance per course session.
Grades management	Manages grade entry, GPA calculation, and transcript generation.
Student management	Manages student profiles, personal records, and academic history.
Billing & payments	Handles invoice generation, payment processing, and financial reporting for students.
Reporting & analytics	Aggregates data from other modules to generate administrative and analytical reports.

5.1.1.2 Important interfaces

Interface	Description
Student → Student Management	Students view and update their personal records and academic history.
Student → Course Management	Students browse the course catalog and submit enrollment requests.
Lecturer → Attendance Management	Lecturers record and review attendance for their courses.
Lecturer → Grades Management	Lecturers submit and manage grades for their students.
Grades Management → Message Queue	Schedules student notifications after grades have been stored successfully.
Grades Management → Email Service	Sends queued grade notifications to students through the external email service.
Administrator → Faculty Management	Administrators manage lecturer accounts and course assignments.
Administrator → Reporting & Analytics	Administrators request and export system-wide reports.
Finance Staff → Billing & Payments	Finance staff manage invoices and verify payment transactions.

Interface	Description
Student Management ↔ SIS	Student records are synchronized with the external Student Information System via REST API.
Billing & Payments ↔ FMS	Financial data is synchronized with the external Financial Management System via REST API.

SIS, FMS, and Email Service are defined as external systems in the [Context and Scope](#).

5.2 Level 2 — Student Management

5.2.1 White-box: Student Management module

The Student Management module follows a layered architecture, separating HTTP handling, business logic, and data access into distinct components. This structure is consistent across all other modules in the system.

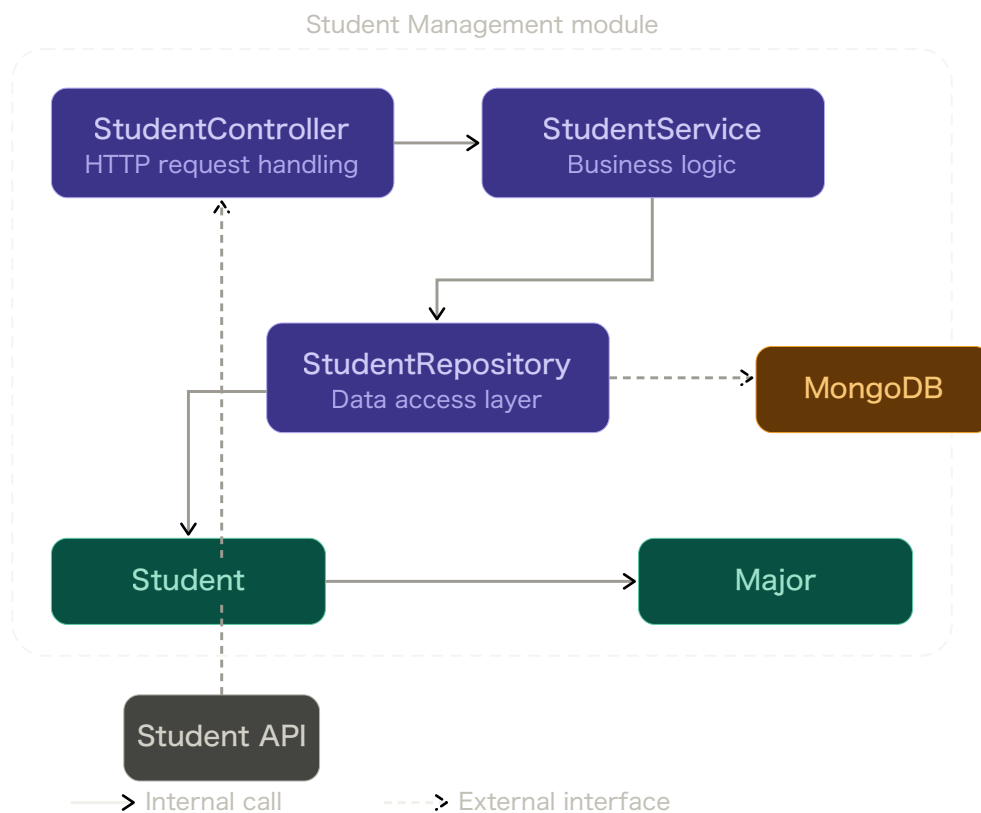


Figure 5.2: Level 2 Building Block View — Student Management module

5.2.1.1 Contained building blocks

Building Block	Responsibility
StudentController	Receives and validates incoming HTTP requests from the Student API. Delegates processing to the StudentService.
StudentService	Contains all business logic for student operations, including validation rules and coordination between components.
StudentRepository	Handles all database interactions. Translates between domain objects and queries against the MongoDB database.
Student	Domain model representing a student, including their id, name, and associated major.
Major	Domain model representing an academic major with abbreviation, name, and description.

5.2.1.2 Important interfaces

Interface	Description
Student API → StudentController	External REST API calls from the frontend or other modules enter through the controller.
StudentController → StudentService	The controller delegates all business operations to the service layer.
StudentService → StudentRepository	The service requests data reads and writes through the repository.
StudentRepository → MongoDB	The repository communicates with the MongoDB database.

5.3 Level 3 — StudentController, StudentService, StudentRepository

The internal structure of the Student Management module's three layers is documented in the class diagram below. Each layer exposes the same set of operations, enforcing a clean separation of concerns from HTTP handling down to data persistence.

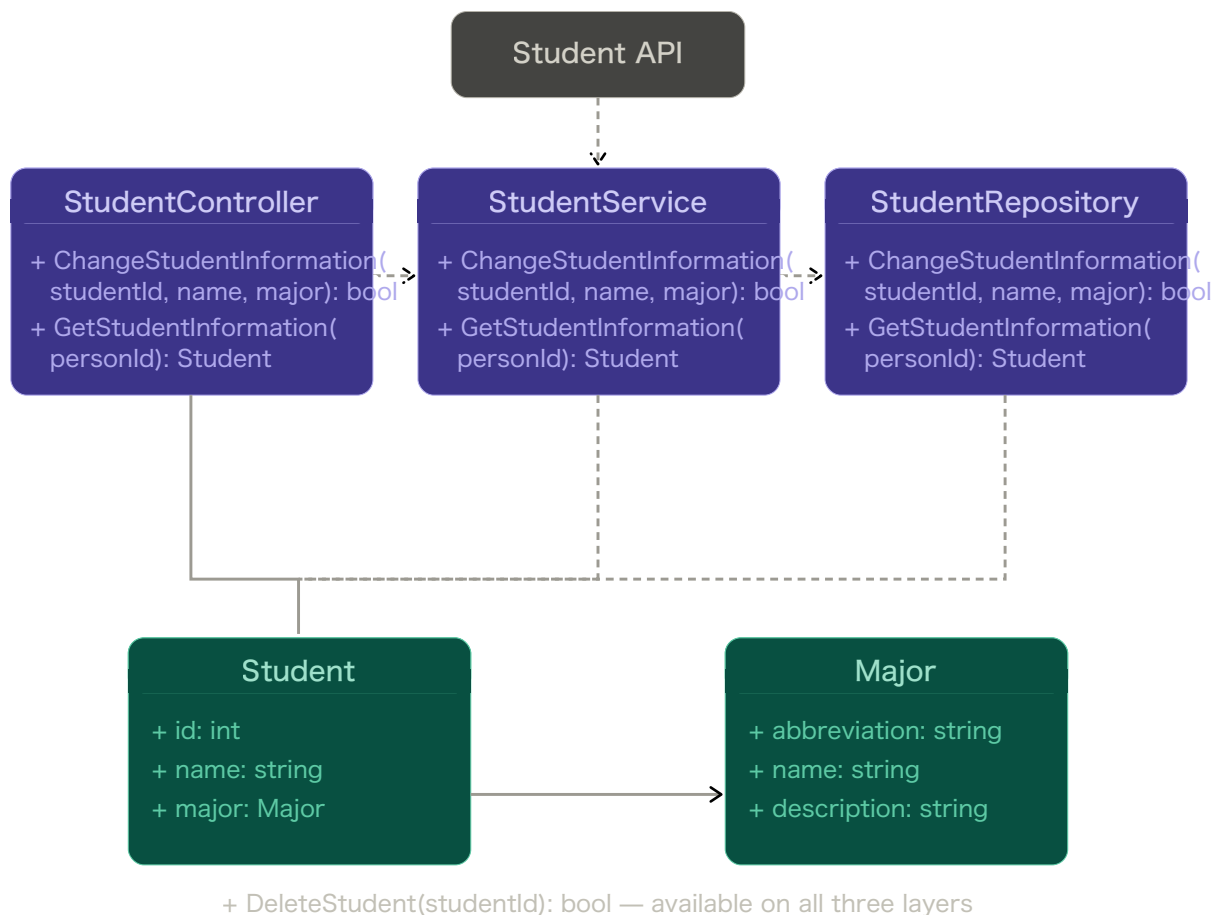


Figure 5.3: Level 3 Class Diagram — Student Management layers

5.3.0.1 Exposed operations

Operation	Description
ChangeStudentInformation(studentId, name, major)	Updates the name and major of an existing student record. Returns a boolean indicating success.
GetStudentInformation(personId)	Retrieves the full Student object for a given person identifier.
DeleteStudent(studentId)	Removes a student record from the system. Returns a boolean indicating success.

6 Runtime View

This runtime view documents two failure scenarios that are important for the architecture:

1. a grade notification cannot be delivered immediately;
2. the result of a payment request is lost during communication with the external FMS.

The scenarios use building blocks from the [Building Block View](#). The Message Queue is a technical communication mechanism used for delayed notifications.

6.1 Runtime Participants

Runtime Participant	Mapping to Section 5	Responsibility
Authentication	Authentication	Verifies the identity, role, and permissions of the user
Grades Management	Grades management	Validates and stores grades, schedules notifications, and processes retries
Grades Collection	Collection in the shared MongoDB database	Stores grade records managed by Grades Management
Message Queue	Technical communication mechanism	Keeps notification messages until they can be processed
Email Service	External system connected to Grades Management	Delivers email notifications to students
Billing & Payments	Billing & payments	Creates payment requests, manages their status, and checks unresolved payments
Payments Collection	Collection in the shared MongoDB database	Stores payment records, request IDs, and status managed by Billing & Payments
FMS	External system connected through the Billing & Payments interface	Processes payments and returns their status

The UMS uses one shared MongoDB database with separate collections for the functional modules. The collections follow the data ownership principle from [Section 8.4](#): grade data is managed by Grades Management and payment data by Billing & Payments. Other modules do not modify these collections directly.

6.2 Grade Notification Cannot Be Delivered

6.2.1 Initial Situation

A lecturer publishes grades for a course. Grades Management stores the grades and schedules an email notification for the students.

The email connection may be unavailable after the grades have already been stored. Repeating the complete grade submission would be wrong because the grade data is already correct.

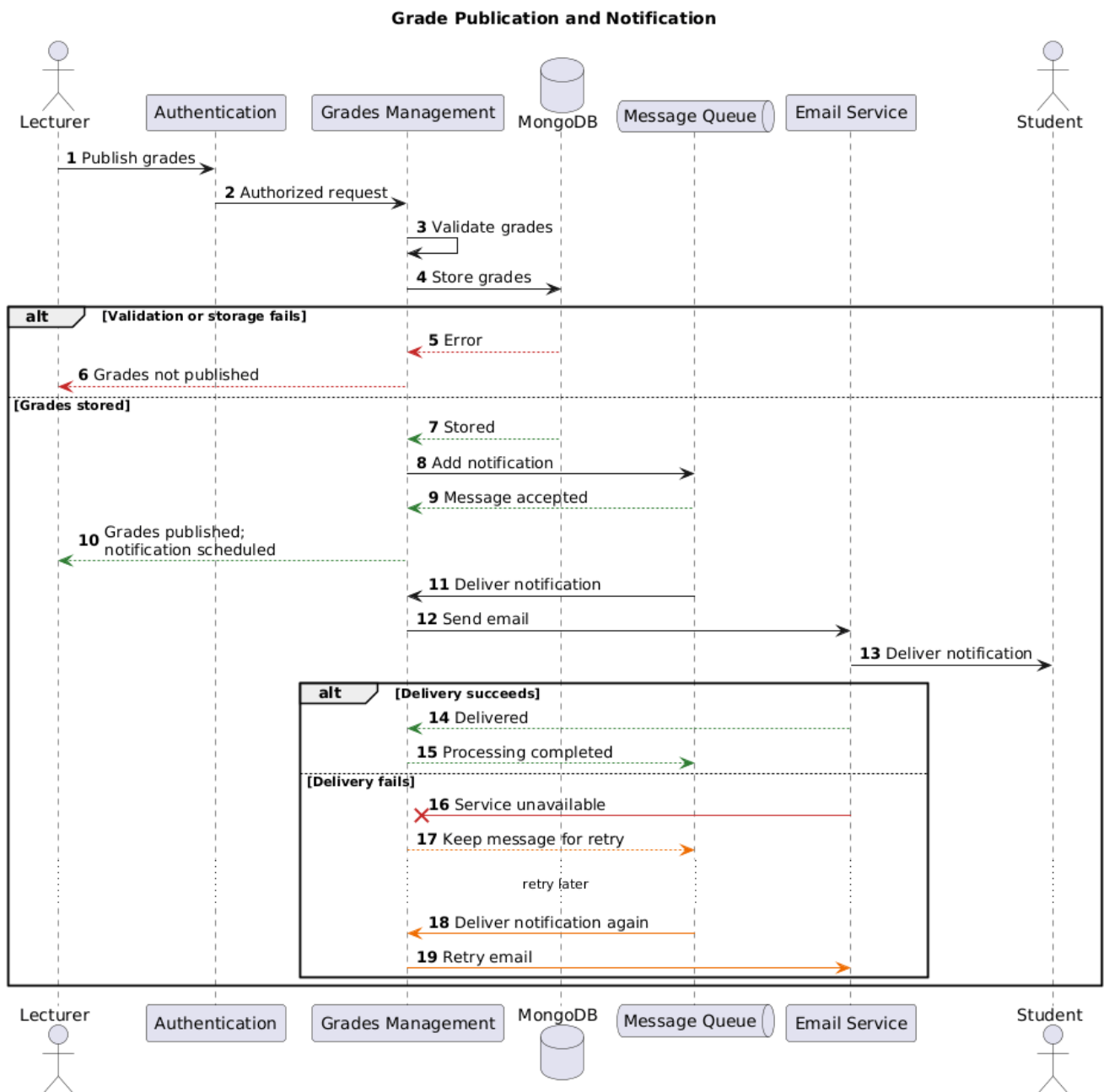


Figure 6.1: Grade publication with delayed notification

6.2.2 Runtime Steps

1. **Authentication** verifies that the lecturer may publish grades for the course.
2. **Grades management** validates and stores the grades.
3. If storage fails, no notification is created and the lecturer receives an error.
4. After successful storage, **Grades management** adds a notification to the Message Queue.
5. The lecturer is informed that the grades were published and the notification was scheduled.
6. Grades Management processes the queued message and sends the notification through the external Email Service.

7. If the Email Service is unavailable, the message remains in the queue and Grades Management processes it again later.

6.2.3 Architectural Decision

Grade storage and notification delivery are separated. Grades Management is responsible for the grade data and sends notifications through the external Email Service. The Message Queue ensures that a temporary email failure does not lose the notification.

The response to the lecturer means:

The grades were stored and the notification was scheduled.

It does not mean that every student has already received an email.

6.2.4 Expected Result

- A failed grade update creates no notification.
- Stored grades remain available if email delivery fails.
- The notification is retried without storing the grades again.
- A failure of the email connection does not block Grades Management.

6.3 Payment Result Is Unknown

6.3.1 Initial Situation

A student starts a payment. Billing & Payments stores the operation and sends a request to the external Financial Management System.

The FMS may process the payment, but the network connection can fail before its response reaches the UMS. The UMS then does not know whether the payment succeeded.

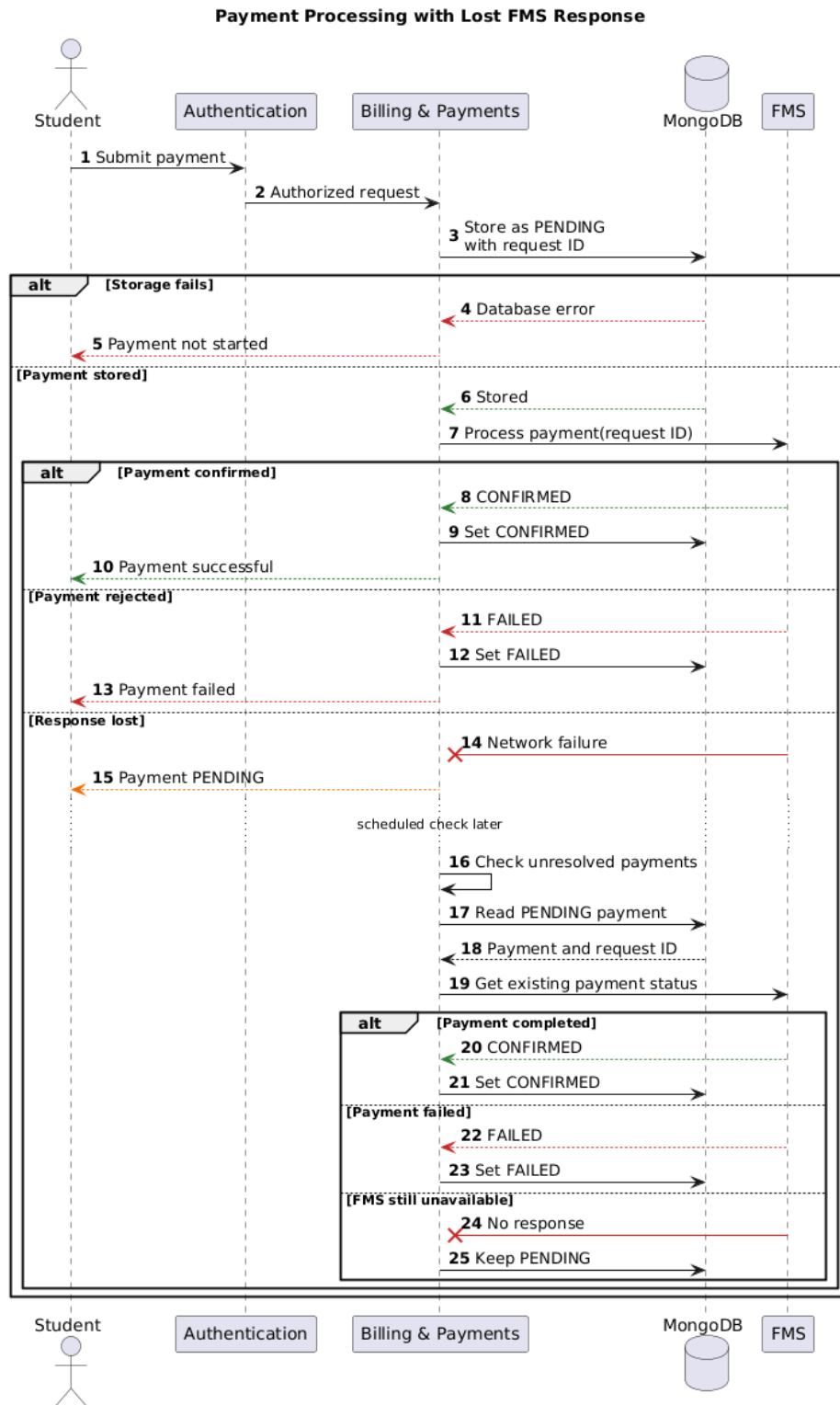


Figure 6.2: Payment processing with a lost FMS response

6.3.2 Runtime Steps

1. **Authentication** verifies the student.
2. **Billing & payments** stores the payment as **PENDING** with a unique request ID.
3. If the payment cannot be stored, the operation stops and no request is sent to the FMS.
4. After successful storage, the payment request is sent to the FMS.

5. A normal FMS response changes the status to `CONFIRMED` or `FAILED`.
6. If the response is lost, the payment remains `PENDING` and the student is informed that it is being verified.
7. Billing & Payments later asks the FMS for the status of the existing request.
8. Billing & Payments stores `CONFIRMED` or `FAILED`. If the FMS is still unavailable, it keeps `PENDING` and tries again later.
9. The student can request and view the current payment status.

6.3.3 Architectural Decision and CAP

CAP is not the main concern between the internal modules of the modular monolith. It is relevant at the network boundary between the UMS and the independently deployed FMS.

During a network failure, the UMS cannot provide both an immediate final response and guaranteed correct payment information. Billing & Payments therefore prioritizes consistency:

- it does not display an unconfirmed payment as successful;
- it keeps the payment in a controlled `PENDING` state;
- it checks the existing payment later using the same request ID instead of creating a second payment.

6.3.4 Expected Result

- An unknown payment result is never shown as successful.
- The payment is not submitted a second time.
- The student sees the clear temporary status `PENDING`.
- The result eventually becomes `CONFIRMED` or `FAILED`.
- If the FMS remains unavailable, the payment stays `PENDING` and is checked again later.

6.4 Summary

Scenario	Actual Risk	Responsible Building Block	Architectural Response
Grade notification	Grades are stored, but notification delivery fails	Grades management	Message Queue retains the notification for a later retry
Payment processing	The FMS response is lost and the payment result is unknown	Billing & payments	Store <code>PENDING</code> and retrieve the existing payment status later

7 Deployment View

7.1 Infrastructure Level 1

7.1.1 Overview Diagram

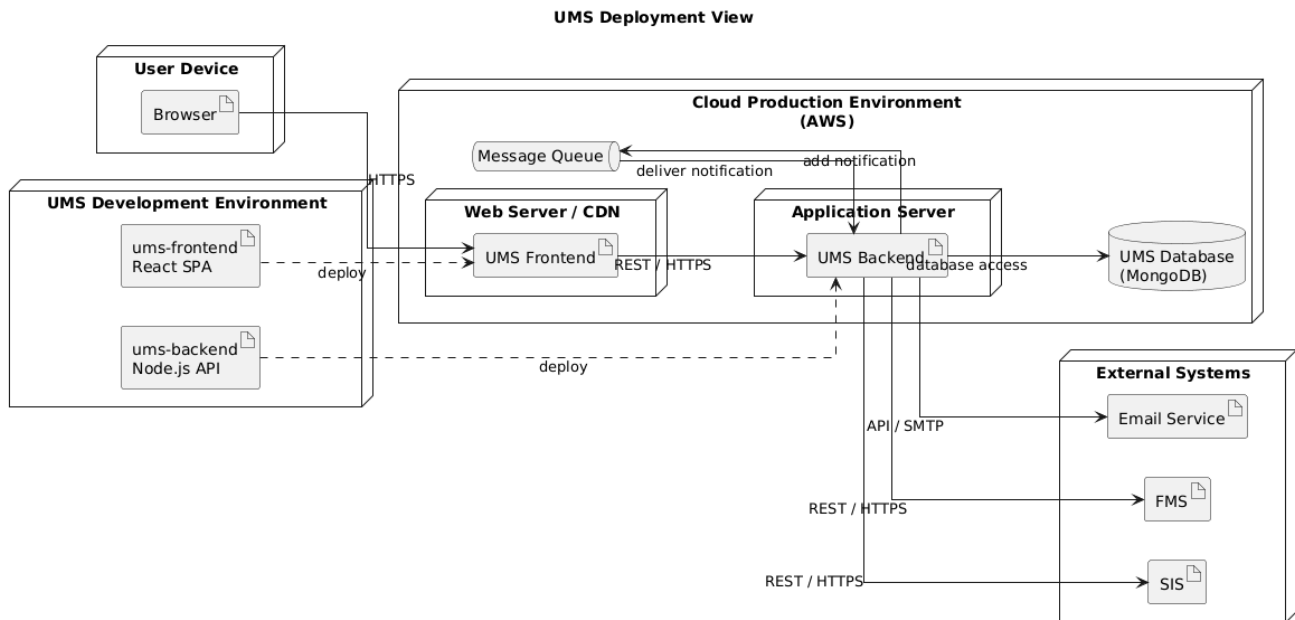


Figure 7.1: Deployment View of the University Management System

7.1.2 Motivation

The University Management System is deployed as a cloud-hosted web application.

Users access the system through a standard web browser. The frontend is implemented using React and communicates with the backend through REST APIs. The backend is implemented using Node.js and contains the business logic for enrollment, attendance tracking, grading, billing, and reporting.

Persistent data is stored in a MongoDB database. Existing university systems such as the Student Information System (SIS) and the Financial Management System are integrated through dedicated APIs.

A Message Queue is used for asynchronous grade notifications. It stores notification messages until Grades Management can process them successfully.

The deployed elements implement the modules from the [Building Block View](#). The queue-based notification and FMS recovery flows are explained in the [Runtime View](#).

This deployment structure supports centralized data management, simple maintenance, and easy access for all user groups.

7.1.3 Quality and/or Performance Features

- Web-based access without client installation.

- Centralized deployment simplifies maintenance and updates.
- Cloud infrastructure supports scalability for growing numbers of users.
- Separation between frontend, backend, and database improves maintainability.
- API-based integration enables reliable communication with external university systems.

7.1.4 Mapping of Building Blocks to Infrastructure

This table maps the logical elements from [Section 5](#) to their physical runtime infrastructure.

Building Block	Infrastructure Element
User Interface	React Web Client
Authentication & Authorization	Node.js Backend
Student Management	Node.js Backend
Course Management	Node.js Backend
Attendance Management	Node.js Backend
Grades Management	Node.js Backend
Billing & Payment Management	Node.js Backend
Reporting & Analytics	Node.js Backend
Academic and Administrative Data	MongoDB Database
Student Information Integration	External SIS
Financial Data Integration	Financial Management System
Email Notification Delivery	External Email Service
Grade Notification Messages	Cloud Message Queue

7.2 Infrastructure Level 2

7.2.1 Application Server

The application server hosts the Node.js backend and provides the business functionality of the system.

Responsibilities:

- authentication and authorization
- enrollment processing
- attendance tracking
- grade management
- billing workflows

- reporting and analytics
- communication with external systems
- processing queued grade notifications
- checking unresolved payment states with the FMS

7.2.2 Database Server

The MongoDB database stores all persistent application data.

Stored information includes:

- student records
- course information
- enrollments
- attendance records
- grades and transcripts
- billing information
- reporting data

The database is only accessed through the backend services and is not directly exposed to users.

8 Cross-cutting Concepts

8.1 Authentication and Authorization

All access to the UMS is controlled through a unified authentication and authorization mechanism that applies to every module and every user group.

Authentication is handled via JSON Web Tokens (JWT). When a user logs in, the backend issues a signed JWT that the client includes in every subsequent request. The backend validates the token on each request without requiring server-side session storage, which supports stateless and scalable deployments.

Authorization is implemented using Role-Based Access Control (RBAC). Each user is assigned exactly one role at login. The available roles are:

Role	Description
Student	Can enroll in courses, view grades and attendance, access course materials, and manage personal data.
Lecturer	Can record attendance, submit grades, upload course materials, and view their assigned courses.
Administrator	Can manage users, courses, schedules, and generate reports.
Finance Staff	Can manage billing, process payments, and access financial reports.

Every API endpoint in the Node.js backend declares the minimum required role. Requests from users with insufficient privileges are rejected with HTTP 403 before reaching the business logic.

The React frontend additionally hides UI elements that a user's role does not permit, reducing confusion and accidental navigation to restricted areas. This is a usability measure only — the backend authorization is always the authoritative enforcement point.

8.2 Input Validation and Error Handling

Input validation and consistent error handling are applied uniformly across all modules to ensure data integrity and provide predictable behavior for the frontend.

Validation is performed in two places:

- In the React frontend, forms validate user input before submission to give immediate feedback and reduce unnecessary network requests.
- In the Node.js backend, all incoming data is validated independently of the frontend. Frontend validation is never trusted as the sole check. Invalid requests return HTTP 400 with a structured error response.

Error responses follow a consistent structure across all API endpoints:

```
{
  "status": 400,
  "error": "Validation failed",
  "details": ["Name must not be empty", "Student ID must be a positive integer"]
}
```

Unhandled exceptions are caught by a global exception handler in the Node.js middleware pipeline. This ensures that internal errors never expose stack traces or system details to the client. All unhandled exceptions are logged (see section 8.3) and return HTTP 500 with a generic error message.

8.3 Logging and Monitoring

Logging is applied consistently across all backend components. Every significant system event — incoming requests, authentication attempts, data modifications, integration calls to the FMS, and errors — is recorded in a structured log.

Log entries follow a consistent format including timestamp, log level, affected module, user identifier (where available), and a short description. Personally identifiable information such as names or contact details is not included in log output.

The application is deployed on AWS, which provides built-in monitoring and alerting capabilities. CloudWatch is used to collect logs, track availability, and alert the operations team when error rates or response times exceed defined thresholds.

Log levels used throughout the system:

Level	Usage
INFO	Normal operations: logins, data reads, successful transactions
WARN	Recoverable issues: failed login attempts, slow queries, FMS API retries
ERROR	Failures requiring attention: unhandled exceptions, FMS integration failures, database errors

8.4 Data Persistence and Consistency

All persistent application data is stored in a centralized MongoDB database. The following principles apply across all modules:

Collection ownership: Each functional module manages its own MongoDB collections. Modules do not directly access collections owned by other modules. Instead, data is exchanged through the responsible service layer, preserving loose coupling between business domains.

Data consistency within the UMS: Changes that affect multiple collections inside MongoDB are executed in a transaction where necessary. If one database operation fails, the transaction is rolled back so that no partial update remains.

External operations: Calls to external systems such as the FMS cannot be part of the same MongoDB transaction. The UMS therefore stores a controlled intermediate state such as `PENDING` before sending the request. If the FMS response is lost, the internal data is not rolled back and the payment is not submitted again. Instead, Billing & Payments later requests the status of the existing operation using its request ID. Course enrollment that depends on payment confirmation is completed only after the payment status is `CONFIRMED`.

The application of these principles to grade publication and external payment processing is shown in the [Runtime View](#).

Data protection: The MongoDB database is accessible only from the Node.js backend within the cloud infrastructure. It is not directly exposed to external users. Database credentials are managed through secure environment variables and are never stored in the application source code.

8.5 Scalability and Deployment

The UMS is deployed on AWS to meet the requirement of supporting high numbers of concurrent users, particularly during peak periods such as course enrollment phases and examination periods.

The corresponding infrastructure is documented in the [Deployment View](#).

The deployment architecture supports horizontal scaling: multiple instances of the Node.js backend can run in parallel behind a load balancer. Because authentication is stateless (JWT, see section 8.1), no session synchronization between instances is required.

The React frontend is delivered as static content, allowing efficient distribution and reducing the load on the application servers.

MongoDB can be deployed as a managed cloud database service, providing automated backups, replication, and high availability. The cloud infrastructure additionally provides health checks, automatic recovery of failed instances, and resource scaling based on system demand.

9 Architecture Decisions

This section documents the most important architectural decisions that have a significant impact on the structure, maintainability, and long-term evolution of the University Management System. Each decision was made after considering alternative approaches and evaluating them against the project's [requirements](#) and [constraints](#).

Problem	Considered Alternatives	Decision
The system should support multiple university processes within a single application while remaining maintainable and easy to extend.	Microservices, Service-Oriented Architecture (SOA), Modular Monolith	A modular monolith was selected because the business domains are closely related and developed by a relatively small team. This approach provides clear separation of concerns while avoiding the operational complexity of distributed systems.
The application must exchange data with the existing Student Information System (SIS) and Financial Management System (FMS).	Direct database access, file import/export, REST API integration	REST API integration was chosen because it provides a loosely coupled and well-defined communication interface. It simplifies maintenance and allows external systems to evolve independently.
Different user groups require different permissions and access rights.	Shared access, manually assigned permissions, role-based access control	Role-Based Access Control (RBAC) was chosen because it offers a clear and scalable way to manage permissions for students, lecturers, administrators, and finance staff while supporting the security requirements of the system.

10 Quality Requirements

This section makes the quality goals from [Section 1](#) more concrete.

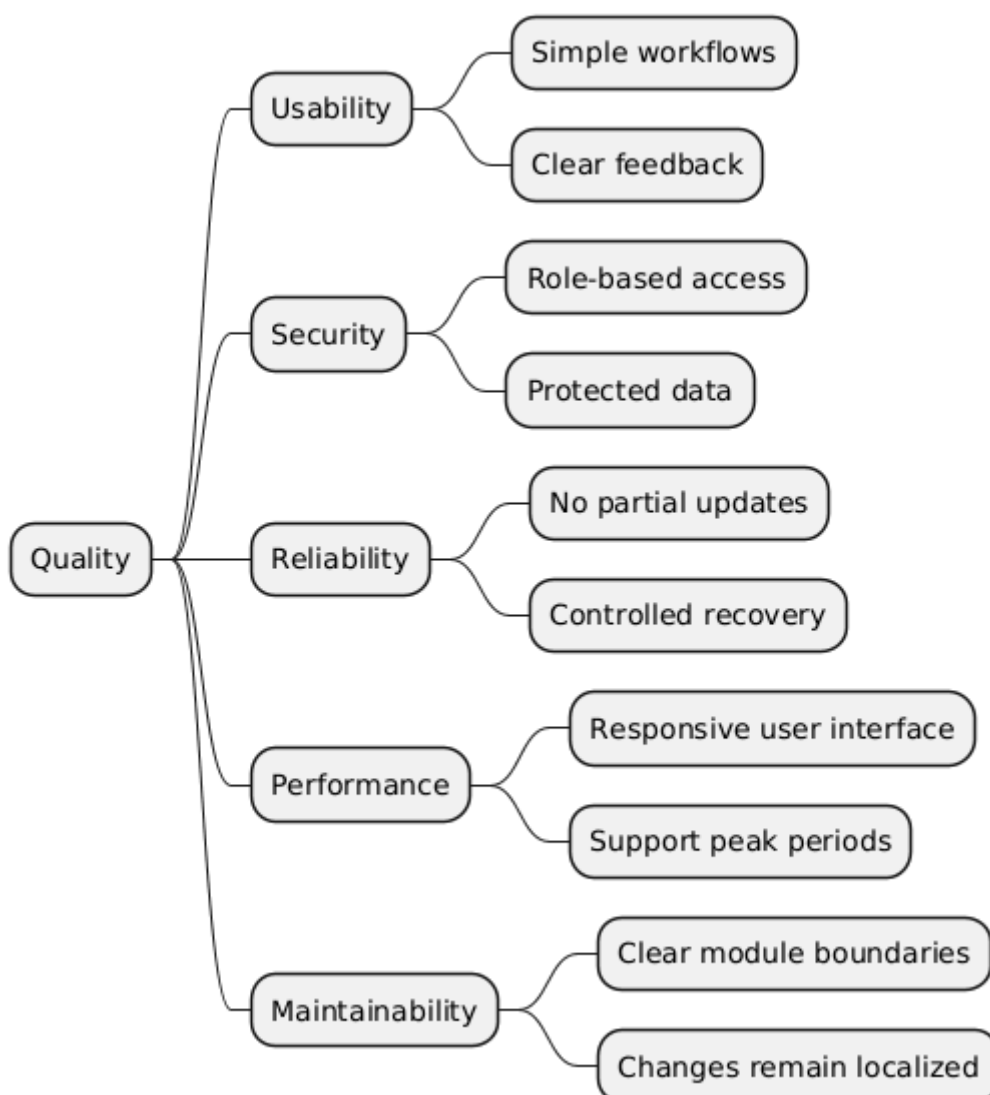
The three prioritized quality goals of the University Management System are:

1. Usability
2. Security
3. Reliability

Performance and maintainability are included in the quality tree as lower-priority attributes because they are already addressed by the cloud deployment and modular monolith.

10.1 Quality Requirements Overview

10.1.1 Quality Tree



Priority	Quality Requirement	Description
1	Usability	The system should be easy to understand and use for students, lecturers, administrators, and finance staff. Common workflows include course enrollment, attendance tracking, grade submission, access to course materials, reporting, and payment processing.
2	Security	Personal student data, grades, attendance records, and financial information must only be accessible to authorized users. Access is controlled through authentication and role-based authorization.
3	Reliability	Important operations such as enrollment, grade publication, attendance tracking, and payment processing must complete correctly or fail in a controlled way without losing or corrupting data.
4	Performance	The browser-based system should remain responsive during normal use and important peak periods such as enrollment.
5	Maintainability	Changes should remain within the responsible module whenever possible.

10.2 Quality Scenarios

The following scenarios are based on the [quality goals from Section 1](#) and on the architectural behavior described in the [Runtime View](#) and [Cross-cutting Concepts](#).

10.2.1 QS-1: Course Enrollment

Field	Description
Scenario ID	QS-1
Scenario Name	Course Enrollment
Source	Student
Stimulus	The student selects an available course and submits an enrollment request.
Environment	Normal operation using the browser-based UMS.
Artifact	React Frontend, Authentication, Course management
Response	The system shows the course information, checks the enrollment requirements, processes the request, and displays either a confirmation or a clear reason for rejection.
Response Measure	After login, the student can complete the enrollment in no more than five user actions without external tools such as spreadsheets or email. The system always displays either a confirmation or a reason for rejection.

10.2.2 QS-2: Unauthorized Access to Grades

Field	Description
Scenario ID	QS-2
Scenario Name	Unauthorized Access to Grades
Source	Student or lecturer without the required permission
Stimulus	A student requests another student's grades, or a lecturer attempts to modify grades for a course they are not assigned to.
Environment	Normal operation with the user logged in to the UMS.
Artifact	Authentication, Grades management, Grades Collection
Response	Authentication and Grades Management reject the request before protected data is returned or modified. The rejected request is logged.
Response Measure	Unauthorized requests must return HTTP 403. No protected grade data may be included in the response and no grade record may be modified.

10.2.3 QS-3: Failure During Grade Publication

Field	Description
Scenario ID	QS-3
Scenario Name	Failure During Grade Publication
Source	Lecturer or technical failure
Stimulus	The lecturer publishes grades, but the grade data cannot be stored or the student notification cannot be delivered.
Environment	Grade publication during normal system operation.
Artifact	Grades management, Grades Collection, Message Queue, Email Service
Response	If grade storage fails, the grades are not published and no notification is created. If storage succeeds but the Email Service cannot deliver the notification, the grades remain stored and the notification stays in the Message Queue for a later retry.
Response Measure	A failed storage operation leaves no partial grade update. A notification is scheduled only after successful grade storage. If delivery fails, the message remains in the queue and is available for the next retry.

10.2.4 QS-4: Lost FMS Response

Field	Description
Scenario ID	QS-4
Scenario Name	Lost FMS Response
Source	Network failure between the UMS and the FMS
Stimulus	The network connection fails after Billing & Payments sends the payment request but before the FMS response reaches the UMS.
Environment	Payment processing during normal system operation.
Artifact	Billing & payments, Payments Collection, FMS
Response	The payment remains <code>PENDING</code> . The UMS does not send the payment again. Billing & Payments later requests the status of the existing payment using its request ID.
Response Measure	An unconfirmed payment is not displayed as successful. A lost response does not create a second payment. The status remains <code>PENDING</code> until the FMS returns <code>CONFIRMED</code> or <code>FAILED</code> .

11 Risks and Technical Debts

This section describes the most important technical risks and debts that result from the chosen [solution strategy](#) and [architecture decisions](#).

The risks are based on the modular monolith, the [integration with existing university systems](#), and the [role-based access control](#) described in the previous sections.

ID	Risk / Technical Debt	Description and Possible Consequence	Mitigation
R-1	Weak module boundaries	The modular monolith can become tightly coupled if modules directly access the logic or MongoDB collections of other modules. Changes in one business area could then affect unrelated parts of the system and make maintenance more difficult.	Define clear module responsibilities and collection ownership. Communication between modules should use their service interfaces. Module dependencies should be checked during code reviews.
R-2	Dependency on SIS and FMS	The UMS depends on the external Student Information System and Financial Management System. These systems may be unavailable, respond slowly, or change their REST APIs. This could prevent student synchronization or leave payment operations unresolved.	Use documented API interfaces, timeouts, controlled error handling, and monitoring. Payment requests with an unknown result remain <code>PENDING</code> and are checked later instead of being repeated.
R-3	Incorrect access permissions	Incorrectly defined roles or missing authorization checks could allow users to access or modify sensitive student, grade, attendance, or financial data.	Enforce RBAC in the backend for every protected endpoint. Add automated authorization tests for every role and log rejected access attempts. The frontend may hide unavailable functions, but it is not the security enforcement point.

The controlled handling of notification and FMS failures referenced by these risks is illustrated in the [Runtime View](#).

11.1 Risk Priority

Priority	Risk	Reason
High	Incorrect access permissions	Unauthorized access could expose or modify sensitive academic and financial data.
High	Dependency on SIS and FMS	External failures can directly affect important student and payment workflows.
Medium	Weak module boundaries	The impact increases over time as the system grows and more modules depend on each other.

12 Glossary

This glossary defines the most important domain-specific and technical terms used in the architecture documentation.

Term	Definition
UMS	University Management System. The web-based system documented in this project.
SIS	Student Information System. An existing external university system that provides official student master data and registration information.
FMS	Financial Management System. An existing external system that processes payments and provides financial transaction information.
Student Master Data	Official information about a student, such as student ID, name, study program, and registration status.
Enrollment	The process of registering a student for a course.
Grade Publication	The process in which a lecturer stores and releases grades so that students can view them.
Modular Monolith	An architecture in which the system is deployed as one application but internally divided into functional modules with clear responsibilities.
Building Block	A part of the system with a defined responsibility, such as Grades Management or Billing & Payments.
RBAC	Role-Based Access Control. Permissions are assigned based on roles such as Student, Lecturer, Administrator, and Finance Staff.
JWT	JSON Web Token. A signed token used by the backend to authenticate requests without storing server-side sessions.
Authentication	The process of verifying the identity of a user.
Authorization	The process of checking whether an authenticated user may access specific data or functionality.
API	Application Programming Interface. A defined interface through which software systems exchange data.
REST API	An HTTP-based interface used for communication between the frontend, backend, SIS, and FMS.
React	A JavaScript library used to implement the browser-based UMS frontend.
Node.js	A JavaScript runtime used to execute the UMS backend application.
HTTPS	An encrypted communication protocol used between web browsers, the UMS, and external REST APIs.

Term	Definition
SMTP	Simple Mail Transfer Protocol. A protocol that can be used to send email notifications.
MongoDB	The database technology used as the central persistent data store of the UMS.
Collection	A group of related documents in MongoDB. Each UMS module manages the collections belonging to its business area.
Message Queue	A technical communication mechanism that temporarily stores messages until they can be processed. The UMS uses it for delayed grade notifications.
Notification	A message sent to a user about an event, such as the publication of a new grade.
PENDING	A temporary payment status indicating that the final result from the FMS is not yet known.
CONFIRMED	A payment status indicating that the FMS has confirmed the payment.
FAILED	A payment status indicating that the payment was rejected or could not be completed.
Request ID	A unique identifier used to recognize a payment request and prevent accidental duplicate processing.
CAP Theorem	A principle for distributed systems stating that during a network partition, a system cannot guarantee both full consistency and full availability.
Network Partition	A communication failure in which the UMS temporarily cannot reach an external system such as the FMS.
Data Ownership	The rule that each functional module manages its own data collections and other modules do not modify them directly.
Cloud Deployment	Operation of the UMS on AWS cloud infrastructure.
AWS	Amazon Web Services. The cloud platform used for hosting and operating the UMS.
CloudWatch	An AWS service used to collect logs, monitor the application, and trigger alerts.
CDN	Content Delivery Network. Infrastructure used to distribute the static React frontend efficiently.
Idempotent Request	A request that can be repeated using the same identifier without creating the same operation more than once.